Taller Práctico: "HeroForge RPG" — Arquitectura de Estado Visual con Redux Toolkit en React Native

---

Asignatura: Desarrollo Móvil

Modalidad: Individual

Stack Tecnológico: React Native (Expo), TypeScript / JavaScript, @reduxjs/toolkit, react-redux

Entregable Principal: Repositorio de código + Video explicativo y demostrativo (Máx. 5 - 7 minutos)

---

1. Propósito Pedagógico: ¿Por qué Redux en un Juego Visual?

En aplicaciones móviles complejas, el estado no pertenece a una sola pantalla ni a un solo componente. Cuando intentamos sincronizar múltiples vistas (cabeceras, pestañas de inventario, barras de progreso, modales de combate y registros de eventos) usando únicamente useState, caemos rápidamente en dos antipatrones críticos:

- **Prop Drilling (Perforación de Props):** Pasar funciones y estados a través de 4 o 5 niveles de componentes que no usan esos datos.
- **Estado Desincronizado y Redundante:** Guardar valores calculados en múltiples estados locales que terminan contradiciéndose en la interfaz gráfica.

Objetivo del Taller: Construir el mini-juego interactivo **"HeroForge RPG"**, donde cada acción despachada (dispatch) en un rincón de la pantalla provoca reacciones visuales

inmediatas en al menos **4 componentes independientes** al mismo tiempo, permitiendo *ver gráficamente* cómo viaja una acción hacia el Store, cómo los Reducers actualizan los Slices y cómo los Selectores recalculan el mundo visual.

---

2. La Experiencia Visual de la App ("HeroForge RPG")

La pantalla de la aplicación estará dividida en **4 Zonas Visuales Independientes** (componentes hermanos sin comunicación por props entre sí). Todos se alimentan exclusivamente del **Redux Store**:

Zona Visual (Componente)	Qué muestra gráficamente	A qué reacciona del Store
Zona A: Avatar & HUD Superior (<HeroAvatarHUD />)	• **Avatar Dinámico:** Cambia de aspecto, color de aura, ícono de arma en mano y expresión según el estado. • **Barras Animadas/Visuales:** Barra de Vida (HP), Barra de Energía/Maná y Barra de Carga de Peso (cambia de Verde a Amarillo y Rojo si hay *Sobrecarga*). • **Badge de Clase**	Lee statsSlice + inventorySlice mediante **Selectores Derivados**. No tiene botones propios; solo reacciona en vivo.

Zona Visual (Componente)	Qué muestra gráficamente	A qué reacciona del Store
	Automática: Etiqueta visual que cambia sola entre *"Aldeano"*, *"Guerrero Berserker"*, *"Archimago"* o *"Tanque Pesado"* según la combinación de stats e ítems.	
Zona B: Panel de Entrenamiento (Atributos) (<StatsControlPanel />)	• Contador de **Puntos de Habilidad Disponibles** (Inicia en 15 pts). • Controles [-] y [+] con barras de nivel para: **Fuerza (STR)**, **Agilidad (AGI)** e **Inteligencia (INT)**. • Botón de *"Poción de Reseteo (Reset)"* y presets rápidos (*"Build Mago"*, *"Build Tanque"*).	Despacha acciones a statsSlice (incrementStat, decrementStat, applyPreset, resetStats).
Zona C: Armería & Mochila (<ArmoryGrid />)	• Cuadrícula de tarjetas visuales de ítems (ej. *Espada de Fuego*, *Escudo de Titanio*,	Despacha acciones a inventorySlice (toggleEquipItem,

Zona Visual (Componente)	Qué muestra gráficamente	A qué reacciona del Store
	Báculo Arcano, *Botas Élficas*, *Armadura Maldita*). ● Cada tarjeta muestra su bono (+Ataque, +Defensa, +Magia) y su **Peso en Kg**. ● Al tocar un ítem se equipa/desequipa con borde brillante. Si el héroe supera el límite de peso, aparece una alerta visual de **"¡SOBRECARGADO! (-50% Velocidad)"**.	unequipAll) y dispara un registro en actionLogSlice.
Zona D: Arena de Simulación & Inspector Redux en Vivo (<ReduxLiveInspector />)	● **Simulador de Combate contra Boss:** Un botón para atacar al Jefe con los stats actuales y recibir daño. ● **Consola Visual de Redux (En pantalla):** Un ticker en la parte inferior que muestra en tiempo real las	Lee los selectores de poder total para el combate y lee actionLogSlice para que el estudiante vea el flujo de Redux en vivo.

Zona Visual (Componente)	Qué muestra gráficamente	A qué reacciona del Store
	últimas 5 acciones despachadas con su type y payload (ej. [inventory/toggleEquipItem] -> "Espada de Fuego").	

---

3. Arquitectura Técnica de Redux Requerida

El proyecto debe estructurarse siguiendo las buenas prácticas de **Redux Toolkit (RTK)** dentro de una carpeta src/store/:

3.1. Los 3 Slices del Store

1. **statsSlice.ts (Atributos y Estado Vital del Héroe):**
    - **Estado inicial:** availablePoints: 15, str: 5, agi: 5, int: 5, currentHp: 100.
    - **Reducers requeridos:**
        - incrementStat(state, action: PayloadAction<'str' | 'agi' | 'int'>): Resta 1 a availablePoints y suma 1 al atributo (solo si hay puntos disponibles).
        - decrementStat(state, action: PayloadAction<'str' | 'agi' | 'int'>): Suma 1 a availablePoints y resta 1 al atributo (ningún atributo base puede bajar de 1).
        - takeDamage(state, action: PayloadAction<number>) y healHero(state): Modifica el HP actual para ver el cambio visual del avatar (herido / derrotado / sano).
        - applyBuildPreset(state, action: PayloadAction<'WARRIOR' | 'MAGE' | 'ROGUE'>): Reasigna automáticamente los 15 puntos en un solo dispatch

para demostrar cómo una sola acción transforma toda la interfaz.

2. **inventorySlice.ts (Armería y Equipamiento):**
    - **Estado inicial:** Catálogo de al menos 6 ítems con propiedades { id, name, icon, category, bonusStr, bonusAgi, bonusInt, weight } y un arreglo equippedIds: string[].
    - **Reducers requeridos:**
        - toggleEquipItem(state, action: PayloadAction<string>): Si el ID ya está en equippedIds, lo quita; si no está, lo agrega.
        - clearEquipment(state): Vacía todo el equipamiento activo de golpe.
3. **actionLogSlice.ts (El "Monitor Visual" de Redux):**
    - **Propósito:** Guardar un historial de los últimos eventos o acciones para mostrarlos en el componente <ReduxLiveInspector /> dentro de la misma app móvil.
    - **Bonus de Arquitectura (Uso de extraReducers):** Pueden capturar acciones de los otros slices usando builder.addMatcher o despachar el registro para que el log se alimente automáticamente cada vez que cambie un atributo o se equipe un arma.

3.2. Regla de Oro: Selectores y Estado Derivado (Prohibido guardar cálculos en el State)

Para demostrar por qué Redux evita inconsistencias, los siguientes valores **NO pueden guardarse como propiedades dentro de los slices**; deben calcularse mediante **Selectores** en un archivo src/store/selectors.ts que combine datos de statsSlice e inventorySlice:

- selectTotalStats: Suma los atributos base (statsSlice) + los bonos de todos los ítems equipados (inventorySlice).
- selectTotalWeight y selectIsOverencumbered: Suma el peso de los ítems equipados y lo compara contra la capacidad de carga máxima (calculada como 10 + (Fuerza Total * 2)). Si supera el límite, devuelve true.
- selectCombatPower: Fórmula combinada de poder total (si está sobrecargado, sufre una penalización del 30% en su poder total).
- selectHeroVisualClass: Retorna la clase visual y el emoji/sprite del avatar según cuál sea su estadística dominante.

---

4. Código Base de Referencia (Esqueleto del Store)

Puedes usar esta base en tu proyecto Expo (instalando previamente npm install @reduxjs/toolkit react-redux):

```
// src/store/index.ts
import { configureStore } from '@reduxjs/toolkit';
import { TypedUseSelectorHook, useDispatch, useSelector } from 'react-redux';
import statsReducer from './slices/statsSlice';
import inventoryReducer from './slices/inventorySlice';
import actionLogReducer from './slices/actionLogSlice';

export const store = configureStore({
  reducer: {
    stats: statsReducer,
    inventory: inventoryReducer,
    actionLog: actionLogReducer,
  },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

// Custom hooks tipados para usar en los componentes visuales
export const useAppDispatch: () => AppDispatch = useDispatch;
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;


// Ejemplo de Selector Cruzado en src/store/selectors.ts
import { RootState } from './index';

export const selectHeroComputedStatus = (state: RootState) => {
  const { str, agi, int, currentHp } = state.stats;
  const { catalog, equippedIds } = state.inventory;

  const equippedItems = catalog.filter(item => equippedIds.includes(item.id));
```

```
  const totalStr = str + equippedItems.reduce((acc, i) => acc + i.bonusStr, 0);
  const totalAgi = agi + equippedItems.reduce((acc, i) => acc + i.bonusAgi, 0);
  const totalInt = int + equippedItems.reduce((acc, i) => acc + i.bonusInt, 0);

  const currentWeight = equippedItems.reduce((acc, i) => acc + i.weight, 0);
  const maxWeightCapacity = 10 + totalStr * 2;
  const isOverloaded = currentWeight > maxWeightCapacity;

  // Determinar apariencia visual dinámicamente
  let heroClass = 'Novato';
  let auraColor = '#94a3b8';
  if (totalStr >= 15) { heroClass = 'Berserker Implacable'; auraColor = '#ef4444';
}
  else if (totalInt >= 15) { heroClass = 'Archimago Arcano'; auraColor = '#3b82f6';
}
  else if (totalAgi >= 15) { heroClass = 'Asesino Sombrío'; auraColor = '#10b981';
}

  const rawPower = (totalStr * 3) + (totalAgi * 2) + (totalInt * 3);
  const combatPower = isOverloaded ? Math.round(rawPower * 0.7) : rawPower;

  return {
    totalStr, totalAgi, totalInt,
    currentWeight, maxWeightCapacity, isOverloaded,
    heroClass, auraColor, combatPower,
    isDefeated: currentHp <= 0
  };
};
```

---

5. Especificaciones del Entregable (Trabajo Individual)

Importante: Este taller es estrictamente **individual**. La evaluación se centra tanto en la implementación funcional del patrón Redux como en tu capacidad para explicar

arquitectónicamente cómo fluyen los datos.

5.1. ¿Qué debe contener la Entrega?

1. **Enlace al Repositorio (GitHub):** Código fuente limpio, con separación clara entre componentes de UI (src/components/) y lógica de estado (src/store/).
2. **Video Explicativo + Demo en Vivo (Duración: 5 a 7 minutos):** El estudiante debe aparecer en cámara o narrar con su voz (compartiendo pantalla con el código y el emulador/dispositivo físico corriendo en simultáneo).

5.2. Guion Obligatorio para el Video (Estructura de 4 Momentos)

- **Minuto 0:00 - 1:30 | El Problema Arquitectónico (Conceptual):**
  - Muestra tu app corriendo y explica: *"¿Qué pasaría si intentáramos conectar el Avatar Superior, la Armería, el Panel de Atributos y la Consola de Eventos usando solo useState y pasando props?"*
  - Explica con tus palabras el ciclo unidireccional de Redux: **UI Component** → **Dispatch(Action)** → **Store/Reducer** → **State Update** → **Selector** → **Re-render visual**.
- **Minuto 1:30 - 3:30 | Demostración Visual en la App (El "Efecto Dominó"):**
  - Interactúa con la app en vivo mostrando cómo **un solo toque** (ej. equipar un ítem pesado o activar un preset de clase) actualiza simultáneamente la barra de peso, cambia la clase/color del Avatar, recalcula el Poder de Combate y registra la acción en el Inspector inferior.
  - Provoca intencionalmente el estado de **Sobrecarga de Peso** y el estado de **Combate/Daño** para evidenciar la reactividad global.
- **Minuto 3:30 - 5:30 | Recorrido por el Código (Slices, Immer y Selectores):**
  - Abre tu editor de código y muestra cómo configuraste configureStore.
  - Elige **un Reducer** (ej. toggleEquipItem o applyBuildPreset) y explica cómo Redux Toolkit maneja la inmutabilidad por debajo (Immer) y qué viaja en el action.payload.
  - Muestra tu archivo de **Selectores** y explica por qué el *Poder de Combate* o la *Sobrecarga* son estados derivados y no variables guardadas en el slice.

- **Minuto 5:30 - 6:30 | Conclusión y Casos de Uso Reales:**
    - Cierra respondiendo brevemente: ¿En qué momento de un proyecto móvil real decidirías usar estado local (useState) y en qué momento escalarías a un Store global como Redux Toolkit?

---

6. Rúbrica de Evaluación (100 Puntos)

Criterio	Peso	Descripción de Excelencia (Nivel Máximo)
1. Arquitectura del Store y Slices (RTK)	**25%**	Configura correctamente el Store con al menos 2-3 slices separados (stats, inventory, actionLog). Utiliza PayloadAction correctamente y mantiene los componentes de UI libres de lógica de negocio pesada.
2. Uso de Estado Derivado (Selectores)	**20%**	Calcula estadísticas totales, límite de peso, sobrecarga y apariencia/clase del héroe cruzando múltiples slices mediante selectores limpios, sin duplicar ni guardar valores calculados dentro

Criterio	Peso	Descripción de Excelencia (Nivel Máximo)
		del estado.
3. Feedback Visual y Reactividad Gráfica	**25%**	La interfaz permite "ver" Redux en acción: barras de progreso dinámicas, cambio de colores/estados del avatar, alertas de sobrecarga y consola visual de acciones despachadas en tiempo real. Cero uso de *prop drilling* entre las 4 zonas.
4. Claridad Técnica en el Video Explicativo	**30%**	El estudiante demuestra dominio propio del código: explica con precisión el flujo *Action → Reducer → Store → Selector*, conecta la explicación teórica con lo que ocurre visualmente en su app y responde con criterio cuándo usar estado local vs. global.